



BURSA TECHNICAL UNIVERSITY

BLM0121 NESNEYE YÖNELİK PROGRAMLAMA DERSİ BAHAR DÖNEMİ PROJE ÖDEVİ

Adı: Muhammed Sabri

Soyadı: CENGİZ

Öğrenci Numarası: 25360859027

GitHub Repo Linki: [GitHub Linki](#)

Youtube Videosu: [YouTube Videosu Linki](#)

Giriş

Bu projede Java dilinin Nesne Yönelimli Programlama prensiplerini kullanarak verilen UML diyagramına ve sınıflar arası kalıtım kurallarına uyarak bize bırakılan bazı kısımları hayal gücü desteğiyle harmanlayarak terminal üzerinden çalışan bir banka otomasyonu geliştirdim.

Kalıtım

Ortak özelliklere sahip nesneler için üst sınıflar tasarlanmıştır. Bankadaki tüm hesapların ortak paydası olan BankaHesabi sınıfı üst sınıf olarak belirlenmiş; VadesizHesap ve YatirimHesabi sınıfları bu sınıftan türetilerek (extends) bakiye ve iban gibi temel özellikleri miras almıştır. Aynı mimari yapı kişiler için de uygulanmış; Kisi adlı üst sınıftan Musteri ve BankaPersoneli alt sınıfları türetilerek ad, soyad gibi fields/bilgilerinin her seferinde baştan yazılması engellenmiştir.

Kapsülleme

Nesnelerin iç durumlarını dış müdahalelerden korumak için tüm sınıf nitelikleri (bakiye, ad, limit vb.) private erişim belirleyicisi ile gizlenmiştir. Bu verilere erişim ve veri atama işlemleri sadece kontrollü getter ve setter metotları üzerinden sağlanmıştır. Örneğin; KrediKarti sınıfında guncelBorc değişkeni dışarıdan setGuncelBorc() metodu ile güncellendiğinde, aynı metot içerisinde kullanılabilirLimit değişkeni arka planda otomatik olarak tekrar hesaplanıp güncellenerek veri tutarlılığı ve sistem güvenliği garanti altına alınmıştır.

Çok Biçimlilik (Polymorphism)

Sınıfların kendine has durumlarını ekrana okunabilir bir formatta basabilmek için, Java'nın kök sınıfından gelen toString() metotları her bir alt sınıf için "Override" (ezilerek yeniden yazma) edilmiştir. Böylece Musteri objesi kendi ID ve iletişim bilgilerini, BankaHesabi objesi IBAN ve bakiyesini, KrediKarti objesi ise güncel borç limit detaylarını sisteme özgü özet çıktılar halinde döndürebilecek yapıya kavuşturulmuştur.

Sistem Mimarisi ve Temel Algoritmalar

Dinamik Veri Üretimi

Sistemdeki kimliklendirici verilerin (ID, IBAN, Kart Numarası) sabit veya manuel girilmesi engellenmiş, gerçekçi bir otomasyon hedeflenmiştir. Personel ID'leri, 16 haneli rastgele kredi kartı numaraları ve IBAN'lar sınıfların yapıcı metotları çalıştığı anda `java.util.Random` sınıfı kullanılarak sistem tarafından otomatik, dinamik ve benzersiz olarak üretilmektedir.

Güvenli Transfer Algoritması

Hesaplar arası para transferi işlemi, statik bir aktarım yerine kullanıcının karar mekanizmasına bırakılmıştır. Transfer işleminde kaynak hesap ve hedef hesap türleri (Vadesiz veya Yatırım) kullanıcı tarafından dinamik olarak seçilmektedir. İşlemin gerçekleşmesi için algoritma önce `gonderenHesap.getBakiye()` $\geq$ miktar şartını aramakta, sadece yeterli bakiye varsa paranın bir hesaptan düşülüp diğerine eklenmesine izin vererek eksi bakiye hatasının önüne geçmektedir.

Hesap ve Kart Silme Güvenliği: en temel kuralı gereği, içerisinde müşteri varlığı veya borcu bulunan hesapların kapatılmasına engel olan güvenli silme geliştirilmiştir. Bir hesabın bakiye değeri 0'dan büyükse hesabın silinmesine sistem tarafından izin verilmemektedir. Aynı şekilde, `guncelBorc` miktarı sıfırlanmamış bir kredi kartının sistemden silinmesi yazılımsal olarak engellenmiştir.

Karşılaşılan Zorluklar ve Çözümler

Proje geliştirme ve test sürecinde bazı mantık ve çalışma zamanı hatalarıyla karşılaştım. Bu süreçte yapay zeka asistanını bir eşli programlama aracı gibi kullanarak hem bu hataları ayıkladım hem de yeni yaklaşımlar öğrenerek projeme entegre ettim.

Sınıf Dönüştürme Hatası (ClassCastException) ve instanceof Kullanımı

- **Zorluk:** Kredi kartı borcu ödeme işlemini test ederken, müşterinin hesaplar listesindeki ilk elemanı (get(0)) doğrudan VadesizHesap sınıfına dönüştürmeye (cast etmeye) çalıştım. Ancak müşteri ilk olarak bir "Yatırım Hesabı" açıtıysa, Java haklı olarak **ClassCastException** hatası fırlattı ve program çöktü.
- **Öğrenilen Çözüm:** Bu sorunu çözerken yapay zeka rehberliğinde Polymorphism (Çok Biçimlilik) mantığının çok önemli bir parçası olan **instanceof** operatörünün kullanımını öğrendim. Listeyi sabit bir index (0) ile zorlamak yerine, bir for döngüsü ile tüm listeyi tarayıp if (hesap instanceof VadesizHesap) kontrolü yaparak uygulamanın çökmesini engelledim ve güvenli veri tipini bulmayı kavradım.

Büyük/Küçük Harf Duyarlılığı (Case Sensitivity) Mantık Hatası

- **Zorluk:** Hesap açılışı sırasında kullanıcının girdiği değeri **startsWith("V")** ile kontrol ediyordum. Test sırasında terminale kazara "vadesiz" (küçük v ile) girdiğimde sistem bunu algılayamıyor ve varsayılan olarak "Yatırım Hesabı" açarak zincirleme bir mantık hatasına (logic bug) sebep oluyordu.
- **Öğrenilen Çözüm:** Girdiyi if bloklarıyla kontrol etmeden önce arka planda, C dilinde de kullandığıma benzer bir mantıkla çalışan **.toUpperCase()** metodu ile veriyi standartlaştırarak bu hatayı kökten çözdüm.

- **Zorluk:** Projenin sonlarına doğru 9. menüde sistemi ve müşteri bilgilerini özet olarak terminale yazdırmak istedim. Ancak `System.out.println(aktifMusteri);` kodunu çalıştırdığımda, ekranda müşterinin adı veya bakiyesi yerine `com.bank.app.Musteri@15db9742` gibi anlamsız yazılar ve sayılar çıktı. Verilerin sisteme doğru kaydedildiğinden emindim ama ekranda neden böyle garip bir şifre çıktığını başta anlayamadım.
- **Öğrenilen Çözüm:** Yapay zekaya bu garip çıktıyı sorduğumda, Java'da bir objeyi doğrudan yazdırdığımızda onun içindeki verileri değil, bellekteki (RAM) referans adresini yazdırdığını öğrendim. Bu sorunu çözmek için, Java'nın kendi içinde var olan varsayılan `toString()` metodunu sınıflarımın içinde "Override" etmeyi (yeniden yazarak ezmeyi) öğrendim. Tüm sınıflara kendi `toString()` metodumu ekledikten sonra o karmaşık referans kodlarından kurtulup temiz ekran çıktıları alabildim.

Ekran Çıktıları ve Test Senaryoları

Menü Yapısı

- ❖ Sistemin ana kontrol merkezi olan etkileşimli arayüz.Switch-case yapısı kullanarak kullanıcının yapabileceği tüm işlemler listelenmiş ve hatalı tuşlamalara karşı giriş koruması sağlanmıştır.

```
SİSTEM : Personel Ahmet Yılmaz sisteme kaydedildi.
* BANKA OTOMASYON SİSTEMİ *

--- İŞLEM MENÜSÜ ---
1. Yeni Müşteri Oluştur
2. Müşteri İçin Hesap Aç
3. Hesaba Para Ekle
4. Hesaplar Arası Para Transferi
5. Kredi Kartı Tanımla
6. Kredi Kartı Borcu Öde
7. Hesap Sil
8. Kredi Kartı Sil
9. Sistem ve Personel Özeti
0. Çıkış
Seçiminiz: |
```

Müşteri oluşturma

- ❖ Sisteme yeni müşteri kaydının yapılması, müşteri numarasının (ID) otomatik üretilmesi ve müşterinin **BankaService** katmanı üzerinden doğrudan bir banka personeline referans olarak atanması.

```
Seçiminiz: 1
Müşteri Adı: Sabri
Müşteri Soyadı: Cengiz
SİSTEM : Sabri Cengiz banka sistemine kaydedildi.
SİSTEM : CSTMR35350 numaralı müşteri, Ahmet isimli temsilciye başarıyla atandı.
SİSTEM: Sabri Cengiz adına müşteri kaydı yapıldı.

--- İŞLEM MENÜSÜ ---
1. Yeni Müşteri Oluştur
2. Müşteri İçin Hesap Aç
3. Hesaba Para Ekle
4. Hesabın Bilgilerini Göster
```

Müşteri adına hesap açma

- ❖ Müşterinin talebine göre .toUpperCase() ile büyük/küçük harf koruması sağlanarak Polimorfizm (Çok Biçimlilik) kullanılarak aynı müşteriye iki farklı türde (Vadesiz ve Yatırım) hesap açılması.

```
Seçiminiz: 2
Açılacak tür (Vadesiz/Yatırım): vadesiz
Vadesiz hesap başarıyla açıldı. IBAN: TR51 7726 9891 9452 4558 3715 82
```

Yatırım Hesabına para yatırma/para çekme

- ❖ UML diyagramında istenen **paraEkle** ve **paraCek** metotlarının testi. Kullanıcının listelenen hesaplarından numara ile (1, 2 vb.) seçim yapması sağlanarak son açılan hesap değil de seçilen hesaba yatırıp/çekebilme ve sonrasında bakiyeyi güncelleme ekrana yansıtma.

```
Seiminiz: 3
İřlem Türü Seçin: [1] Para Ekle | [2] Para Çek
Seiminiz: 1

--- Mevcut Hesaplarınız ---
1. Vadesiz Hesap | Bakiye: 1000.0 TL
2. Yatırım Hesabı | Bakiye: 0.0 TL
İřlem yapılacak hesabı seçin (1, 2...): 2
Miktar: 400
400.0 TL hesaba eklendi. Yeni Bakiye: 400.0
```

```
Seiminiz: 3
HATA: İřlem yapılacak hesap bulunamadı!
```

Hesaplar arasında para transferi gerekleřtirme

- ❖ Proje yönergesinde belirtilen 3 parametrelili (aliciHesap, gonderenHesap, miktar) transfer metodunun alıřtırılması. Kaynak hesapta yeterli bakiye olup olmadığı kontrol edilerek, bakiye yetersizse hata mesajı verilmiř, yeterliyse Vadesiz ve Yatırım hesapları arasında güvenli para aktarımı saėlanmıřtır.

```
Seiminiz: 4
HATA: Transfer için en az 2 hesap gereklidir!
```

```
Alıcı hesap, gönderen hesap ve miktar parametre olarak alınır
public void paraTransferi(BankaHesabi aliciHesap, BankaHesabi gonderenHesap, double miktar)
{
    if (miktar > 0 && gonderenHesap.getBakiye() >= miktar) {
        // Gönderen hesabın bakiyesinden miktar düşölür
        gonderenHesap.setBakiye(gonderenHesap.getBakiye() - miktar);
        // Alıcı hesabın bakiyesine miktar eklenir
        aliciHesap.setBakiye(aliciHesap.getBakiye() + miktar);
        System.out.println("Transfer Başarılı! Gönderilen: " + miktar + " TL");
    } else {
        System.out.println("HATA: Transfer için yetersiz bakiye veya geçersiz tutar!");
    }
}
```

```
Seiminiz: 4
Kaynak Hesap (Vadesiz/Yatırım): vadesiz
Hedef Hesap (Vadesiz/Yatırım): yatırım
Miktar: 400
Transfer Başarılı! Gönderilen: 400.0 TL
-> Kaynak Hesabın Güncel Bakiyesi: 100.0 TL
-> Hedef Hesabın Güncel Bakiyesi: 400.0 TL
```

Müşteriye kredi kartı tanımlama

- ❖ Müşteriye yeni bir kredi kartı tanımlanması işlemi. String.format ve java.util.Random sınıfı kullanılarak 16 haneli (4-4-4-4 boşluklu okunabilir yapıda) dinamik bir kart numarası sistem tarafından üretilmiştir.

```
private String generateRandomCardNumber() {
    java.util.Random rnd = new java.util.Random();
    //4 yerine 1-9 arası rastgele bir ilk hane üretiyoruz
    StringBuilder sb = new StringBuilder(String.valueOf(rnd.nextInt(9) + 1));
    // İlk hane yukarıda üretildiği için döngü 1'den başlıyor
    for (int i = 1; i < 16; i++) {
        if (i % 4 == 0) { |
            sb.append(" "); // 4-4-4-4 formatı için aralara boşluk
        }
        sb.append(rnd.nextInt(10)); // Kalan 15 hanevi random dolduruyoruz.
    }
}
```

```
Seçiminiz: 5
Kredi Kartı Limiti ne kadar olsun?: 20000
Kredi kartı tanımlandı. Kart No: 4889 2713 7752 8343
SİSTEM: Test işlemleri için karta 9769.0 TL yapay borç yansıtıldı.

--- İŞLEM MENÜSÜ ---
1. Yeni Müşteri Oluştur
2. Müşteri İçin Hesap Aç
3. Müşteriye Borç Ekle
```

Kredi kartı borcu ödeme işlemi yapma

- ❖ **Random** sınıfı ile dinamik (limiti aşmayacak şekilde) test borcu üretilerek karta yansıtılması ve ardından Vadesiz hesap bakiyesi kullanılarak kredi kartı borcunun güvenli bir şekilde ödenmesi.
- ❖ Kredi kartı borcu ödeme işleminde, yönergede ve UML diyagramında istenen **krediKartiBorcOdeme(KrediKarti kart, double miktar)** parametre yapısına harfiyen uyulmuştur. Ancak konsol üzerinden işlem yapan kullanıcıdan 16 haneli kart numarasını manuel olarak girmesini beklemek hata riskini artıracığı için, sistem arka planda otomatize edilmiştir. Kullanıcının aktif kredi kartı nesnesi, Müşteri sınıfındaki ArrayList içerisinde dinamik olarak çekilerek (aktifMusteri.getKrediKartlari().get(0)) metoda doğrudan obje referansı olarak geçirilmiş ve parametre şartı kullanıcıya yansıtılmadan güvenli bir şekilde sağlanmıştır.

```
Seiminiz: 6
Güncel Borcunuz: 9769.0 TL
Ödenecek Bor Miktarı: 400
HATA: Ödeme için yetersiz bakiye!
```

```
Seiminiz: 6
HATA: Kredi kartı borcu ödemek için sistemde bir 'Vadesiz Hesap' açmış olmanız gereklidir!
```

```
Seiminiz: 6
Güncel Borcunuz: 9769.0 TL
Ödenecek Bor Miktarı: 90
Kart borcu ödendi. Kalan Bor: 9679.0
```

Gerekli şartlar sağlandığında hesap silme işlemi gerçekleştirme

- ❖ İçerisinde bakiye olan hesabın, bakiyesi transfer edilmeden silinmesine izin verilmeyen güvenlik mekanizmasının testi.

```
Seiminiz: 7
SİSTEM: 800.0 TL bakiye başarıyla istenilen diğer hesaba aktarıldı.
Hesap başarıyla silindi: TR44 4677 7785 7248 3293 8490 59
```

Gerekli şartlar sağlandığında kart silme işlemi gerçekleştirme

- ❖ Güncel borcu sıfırlanmamış bir kredi kartının sistemden (ArrayList üzerinden) silinmesini engelleyen kuralın ve bor ödendikten sonraki güvenli silme işleminin testi.

```
Seiminiz: 8
Lütfen öncelikle bor ödemesi yapınız. Güncel Bor: 9679.0
```

```
Seiminiz: 6
Güncel Borcunuz: 9679.0 TL
Ödenecek Bor Miktarı: 9679
Kart borcu ödendi. Kalan Bor: 0.0

--- İŞLEM MENÜSÜ ---
1. Yeni Müşteri Oluştur
2. Müşteri İçin Hesap Aç
3. Hesaba Para Ekle
4. Hesaplar Arası Para Transferi
5. Kredi Kartı Tanımla
6. Kredi Kartı Borcu Öde
7. Hesap Sil
8. Kredi Kartı Sil
9. Sistem ve Personel Özeti
0. Çıkış
Seiminiz: 8
Kredi kartı başarıyla silindi: 4889 2713 7752 8343
```

Sistem ve müşteri bilgileri

- ❖ Musteri, BankaPersoneli, BankaHesabi ve KrediKarti sınıflarındaki toString() metodlarının ezilmesi (Override) ile oluşturulan detaylı sistem raporu.

```
=SİSTEM VE MÜŞTERİ ÖZETİ=
-> GÖREVLİ PERSONEL BİLGİSİ:
Ad: Ahmet Soyad: Yılmaz Email: ahmet@banka.com Telefon: 5551122 Personel ID: PRSNL7602 Sorumlu Olduğu Müşteri Sayısı: 1

-> AKTİF MÜŞTERİ BİLGİSİ:
Ad: Sabri Soyad: Cengiz Email: sabri@mail.com Telefon: 5550000 | Müşteri No: CSTMR50221

-> MÜŞTERİ HESAPLARI:
- IBAN: TR96 5603 1205 7340 7689 5831 02 | Bakiye: 10000.0 TL | Hesap Türü: Vadesiz Hesap

-> KREDİ KARTLARI:
- Kart No: 4323 7520 1689 1138 | Limit: 5000.0 | Güncel Borç: 3593.0 | Kullanılabilir: 1407.0
-----
```

```
===== SİSTEM VE MÜŞTERİ ÖZETİ =====
-> GÖREVLİ PERSONEL BİLGİSİ:
Ad: Arda Soyad: Albayrak Email: arda@banka.com Telefon: 5551632 Personel ID: PRSNL9082 Sorumlu Olduğu Müşteri Sayısı: 2

--- BANKA SİSTEMİNDEKİ TÜM MÜŞTERİLER ---
- Müşteri No: CSTMR45903 | İsim: sabri Cengiz
- Müşteri No: CSTMR28157 | İsim: muhammed şimşek

-> AKTİF MÜŞTERİ BİLGİSİ:
Ad: muhammed Soyad: şimşek Email: muhammed@mail.com Telefon: 5550000 | Müşteri No: CSTMR28157

-> MÜŞTERİ HESAPLARI:
- IBAN: TR04 0829 7176 9377 2368 2532 93 | Bakiye: 0.0 TL | Hesap Türü: Yatırım Hesabı
- IBAN: TR39 7008 6638 6694 0901 7638 75 | Bakiye: 15000.0 TL | Hesap Türü: Vadesiz Hesap

-> KREDİ KARTLARI:
- Kart No: 1550 7212 6693 0774 | Limit: 50000.0 | Güncel Borç: 16470.0 | Kullanılabilir: 33530.0
=====
```

Çıkış

- ❖ Konsol kaynaklarının serbest bırakılarak simülasyonun sonlandırılması ve programın güvenli bir şekilde kapatılması.

```
0. Çıkış
Seçiminiz: 0
Sistem kapatılıyor...
```

Sonuç

Bu projeyle Java'da sınıf mimarisi kurmayı, referans tiplerle çalışmayı (ArrayList) ve kullanıcı deneyimi odaklı konsol uygulaması geliştirmeyi daha net kavradım. Kodların sadece hata vermeden çalışmasının değil, kurallara uygun ve birbirine bağlı bir sınıf mimarisiyle yazılmasının ne kadar önemli olduğunu anladığım, oldukça öğretici bir ödev süreci oldu.